

#중급반 3주차

동적 계획법

<Dynamic Programming>

#동적 계획법

<Dynamic Programming>

동적 계획법(Dynamic Programming, DP)은 문제의 재귀적인 구조를 찾아 작은 문제의 답으로 큰 문제의 답을 구하는 문제해결 방식입니다.

특정한 알고리즘이 아니라 사고의 패러다임 중 하나이기 때문에 DP를 이해하려면 사례를 통해 문제를 재귀적으로 쪼개는 법을 익혀야 합니다.

재귀에 대해 간단하게 한번 알아봅시다.

#동적 계획법

<Dynamic Programming>

재귀적 사고에 익숙해지기 위해 재귀함수에 대해
간단히 살펴봅시다.

오른쪽 코드는 N부터 1까지 순서대로 출력하는 함수입니다.

f(5)가 호출되면 5를 출력한 뒤 f(4)를 호출합니다.

f(4)가 호출되면 4를 출력한 뒤 f(3)을 호출합니다.

...

f(1)이 호출되면 1을 출력한 뒤 f(0)을 출력하고,

f(0)이 호출되면 아무것도 하지 않고 종료합니다.

이후 호출 순서대로 f(1), f(2), ..., f(5)가 차례대로 종료됩니다.

따라서 5, 4, 3, 2, 1이 순서대로 출력됩니다.

```
#include <iostream>
using namespace std;

void f(int N){
    if (N == 0)
        return;
    cout << N << '\n';
    f(N-1);
}

int main(){
    f(5);
}
```

#동적 계획법

<Dynamic Programming>

오른쪽 코드는 1부터 N까지 출력하는,
이전 슬라이드의 코드와 반대로 동작하는 코드입니다.

f(5)를 호출하면 f(4)를 호출합니다. 이후 재귀적으로 f(3),
f(2), f(1), f(0)이 호출됩니다.

f(0)이 종료되면 f(1)으로 돌아와 1을 출력하고, f(1)이 종료됩
니다. 이후 f(2)로 돌아와 2를 출력하고 f(2)도 종료됩니다.

이 과정이 반복되어 f(5)까지 종료되면 1, 2, 3, 4, 5가 순서대
로 출력됩니다.

```
#include <iostream>
using namespace std;

void f(int N){
    if (N == 0)
        return;
    f(N-1);
    cout << N << '\n';
}

int main(){
    f(5);
}
```

#동적 계획법

<Dynamic Programming>

일반적으로 재귀 함수는 해결하고자 하는 문제를 재귀적으로
쪼개어 설계합니다.

예를 들어, 1부터 N까지 출력하기는 1부터 N-1까지 출력한 뒤
N 출력하기로 생각할 수 있습니다.

마찬가지로, N부터 1까지 출력하기는 N 출력한 뒤 N-1부터 1
까지 출력하기로 생각할 수 있습니다..

이를 바탕으로 오른쪽처럼 재귀 함수를 설계할 수 있습니다.

```
void print_1_to_N(int N){  
    if (N == 0)  
        return;  
    print_1_to_N(N-1);  
    cout << N << '\n';  
}
```

```
void print_N_to_1(int N){  
    if (N == 0)  
        return;  
    cout << N << '\n';  
    print_N_to_1(N-1);  
}
```

#동적 계획법

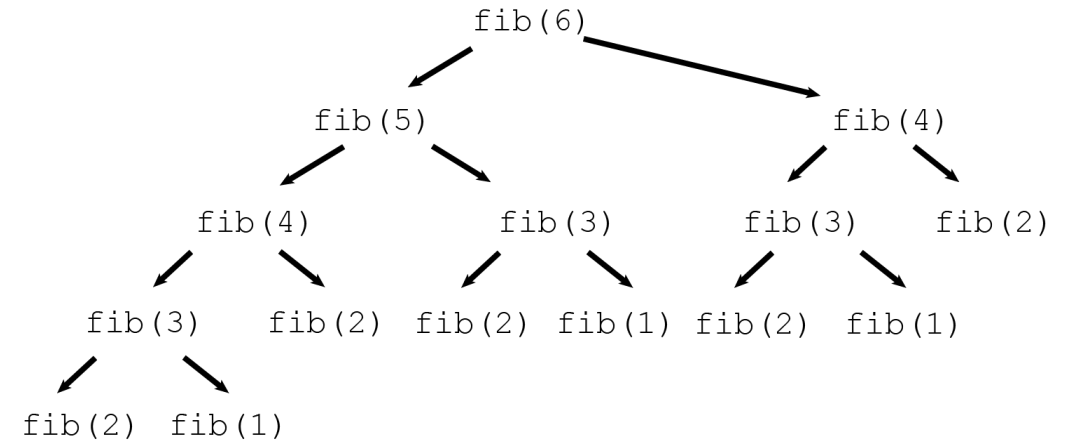
<Dynamic Programming>

재귀적인 구조는 전체 문제를 풀기 위한 부분 문제가 전체 문제와 같은 구조임을 의미합니다.

예를 들어, 피보나치 수의 계산을 생각해 봅시다.

fib(6)은 이전 두 항인 fib(5)와 fib(4)의 값을 더하여 구합니다. 이때, fib(5)와 fib(4)의 값 또한 이전 두 항을 더하는 방식으로 계산됩니다.

재귀적인 구조는 보통 점화식으로 나타낼 수 있기 때문에, 처음 배우는 단계에서는 동적 계획법으로 문제를 푸는 과정을 문제에 대한 점화식을 찾는다고 생각해도 됩니다.



$$fib(n) = \begin{cases} 0 & (n = 0) \\ 1 & (n = 1) \\ fib(n-1) + fib(n-2) & (otherwise) \end{cases}$$

#동적 계획법

<Dynamic Programming>

동적 계획법으로 문제를 푸는 과정은

- 1) 문제를 재귀적인 관계로 나타내기
- 2) 점화식 계산

두 단계로 나눌 수 있습니다.

점화식 계산은 보통 기계적으로 수행할 수 있으니,
문제를 풀어 보며 재귀적인 관계를 세우는 연습을 먼저 한
뒤 점화식 계산하는 법을 배우겠습니다.

3 1, 2, 3 더하기 (BOJ #9095)

<문제 설명>

- 정수 $N(N < 11)$ 이 주어질 때,
1, 2, 3으로만 이루어져 있고 합이 N 인
수열을 만드는 경우의 수 구하기
- 순서가 다르면 다른 경우
ex) $1 + 2$ 와 $2 + 1$ 은 다른 경우

#동적 계획법

<Dynamic Programming>

N = 4일 때의 답을 살펴봅시다. 오른쪽처럼 7가지가 있습니다.

순서가 다르면 다른 경우이니, 첫 번째 수를 기준으로 경우를 나누어 봅시다.
첫 번째 수가 1인 경우는 4가지, 2인 경우는 2가지, 3인 경우는 1가지입니다.

첫 번째 수가 1일 때를 봅시다.

첫 번째 수를 1으로 고정하면 나머지 수들의 합은 3이 되어야 합니다.

따라서, 첫 번째 수가 1인 경우의 수는 1, 2, 3으로 3을 만드는 경우의 수와 같습니다.

- 1 + 1 + 1 + 1
- 1 + 1 + 2
- 1 + 2 + 1
- 1 + 3
- 2 + 1 + 1
- 2 + 2
- 3 + 1

#동적 계획법

<Dynamic Programming>

같은 논리로, 첫 번째 수가 2인 경우는 2를 만드는 경우의 수와,
3인 경우는 1을 만드는 경우의 수와 같습니다.

세 가지 경우의 수를 모두 합치면 4를 만드는 전체 경우의 수가 됩니다.

N을 만드는 경우의 수를 $f(N)$ 이라고 표현하면,

$f(4) = f(3) + f(2) + f(1)$ 과 같습니다.

같은 논리로, $f(5) = f(4) + f(3) + f(2)$ 입니다.

일반화하면, $f(N) = f(N-1) + f(N-2) + f(N-3)$ 이라는 점화식을 얻게 됩니다.

- $1 + 1 + 1 + 1$
- $1 + 1 + 2$
- $1 + 2 + 1$
- $1 + 3$
- $2 + 1 + 1$
- $2 + 2$
- $3 + 1$

#동적 계획법

<Dynamic Programming>

동적 계획법은 작은 문제의 답으로 큰 문제의 답을 구하는 방식이라고 했습니다.

어떤 문제 A를 풀기 위해 B의 답이 필요할 때 B를 작은 문제, A를 큰 문제라고 생각합니다. 즉, **문제의 크기는 문제 사이의 종속성에 따라 정의됩니다.**

예를 들어, 앞서 살펴본 점화식 $f(N) = f(N-1) + f(N-2) + f(N-3)$ 에서 $f(N)$ 을 큰 문제, 나머지를 작은 문제로 봅니다.

2024 ALOHA 2학기 멘토링

5

#동적 계획법
<Dynamic Programming>

동적 계획법(DP)은 문제의 재귀적인 구조를 찾아 작은 문제의 답으로 큰 문제의 답을 구하는 문제해결 방식입니다.

특정한 알고리즘이 아니라 사고의 패러다임 중 하나이기 때문에 DP를 이해하려면 사례를 통해 문제를 재귀적으로 쪼개는 법을 익혀야 합니다.



2024 ALOHA

#동적 계획법

<Dynamic Programming>

점화식을 계산하는 방식은 크게 Top-down 방식과 Bottom-up 방식으로 나뉩니다.

Bottom-up 방식은 반복문을 통해 작은 문제부터 큰 문제 순서로 답을 구해가는 방식이며, Top-down 방식은 재귀 함수를 통해 큰 문제에서부터 작은 문제를 찾아가는 방식입니다.

#동적 계획법

<Dynamic Programming>

오른쪽 코드는 Bottom-up 방식으로 문제를 푸는 코드입니다.

1, 2, 3 더하기 문제의 점화식은 $f(N) = f(N-1) + f(N-2) + f(N-3)$ 이었습니다. 이 점화식은 $N \leq 3$ 일 때는 적용할 수 없으니, $N \leq 3$ 인 경우의 답은 미리 구해 두어야 합니다.

이 문제에서는 N 이 작을수록 작은 문제입니다.

따라서, N 이 작은 경우부터 차례로 답을 계산하면 문제의 답을 구할 수 있습니다.

$f(1), \dots, f(N)$ 을 모두 구해두는 시간복잡도는 자명히 $O(N)$ 입니다.

```
#include <iostream>
using namespace std;

int dp[11];
int main(){
    dp[1] = 1; dp[2] = 2; dp[3] = 4;
    for (int i = 4; i <= 10; i++){
        dp[i] = dp[i-1] + dp[i-2] + dp[i-3];
    }

    int t;
    cin >> t;
    for (int i = 0; i < t; i++){
        int N;
        cin >> N;
        cout << dp[N] << '\n';
    }
}
```

#동적 계획법

<Dynamic Programming>

오른쪽 코드는 Top-down 방식으로 문제를 푸는 코드입니다.
Top-down 방식의 핵심은 계산 결과를 dp 배열에 저장하고,
이미 결과가 계산된 값은 다시 계산하지 않는 것입니다.

이를 메모이제이션이라고 합니다.

```
#include <iostream>
using namespace std;

int dp[11];

int f(int N){
    if (N == 1) return 1;
    if (N == 2) return 2;
    if (N == 3) return 4;
    if (dp[N] != 0) return dp[N];
    dp[N] = f(N-1) + f(N-2) + f(N-3);
    return dp[N];
}

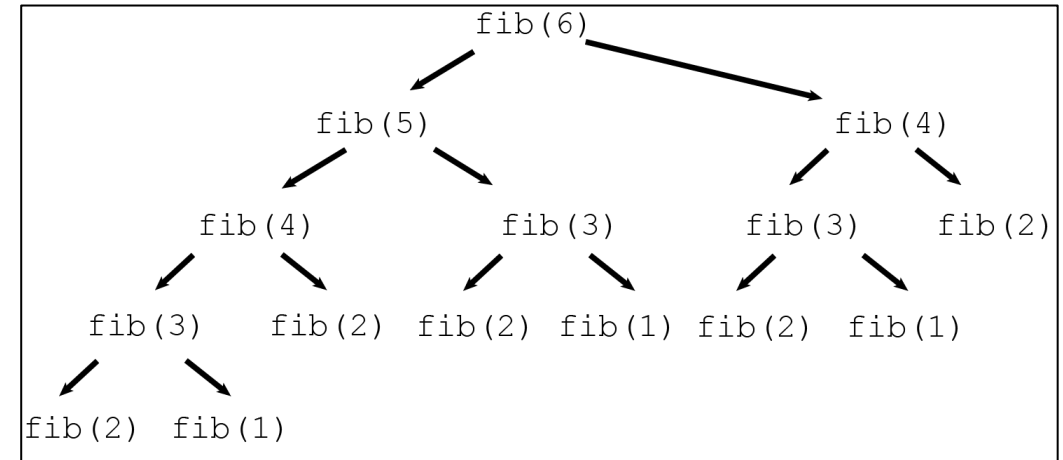
int main(){
    int t;
    cin >> t;
    for (int i = 0; i < t; i++){
        int N;
        cin >> N;
        cout << f(N) << '\n';
    }
}
```

#동적 계획법

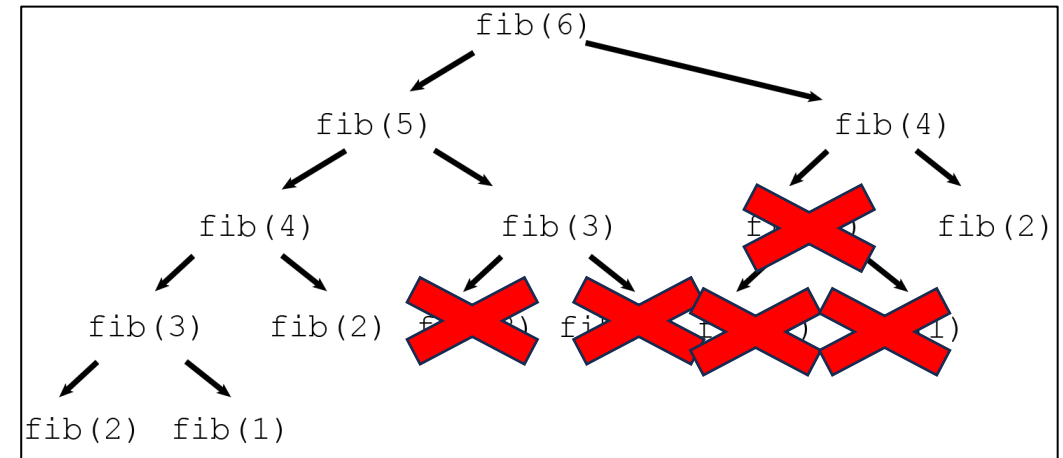
<Dynamic Programming>

위 그림은 피보나치 수를 메모이제이션 없이 단순 재귀로,
아래 그림은 메모이제이션을 수행하여 구하는 방식입니다.

메모이제이션을 수행하지 않으면 이미 계산한 값에 대해서도
계속 재귀 호출을 수행하므로, 필요 없는 계산이 너무 많아집
니다. 이때의 시간 복잡도는 $O(2^N)$ 입니다.



메모이제이션을 적용하지 않았을 때, $O(2^N)$



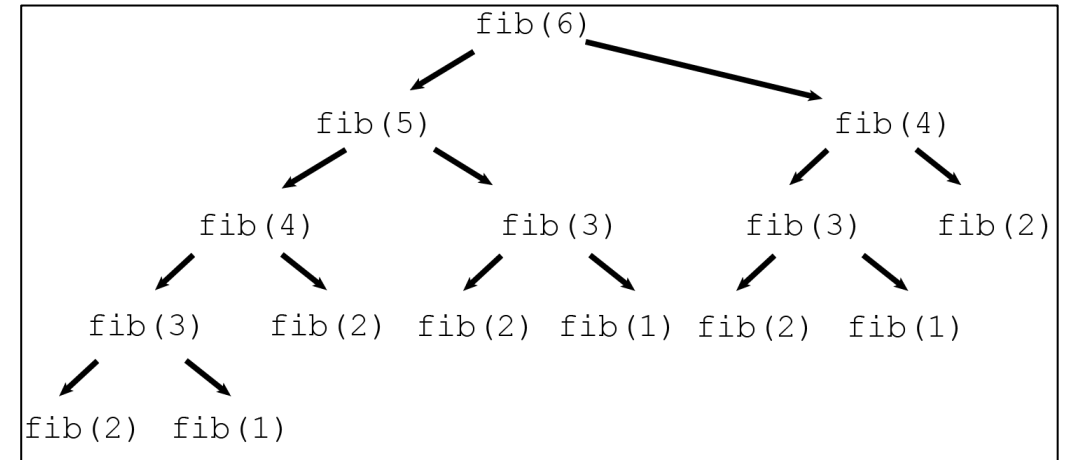
메모이제이션을 적용했을 때, $O(N)$

#동적 계획법

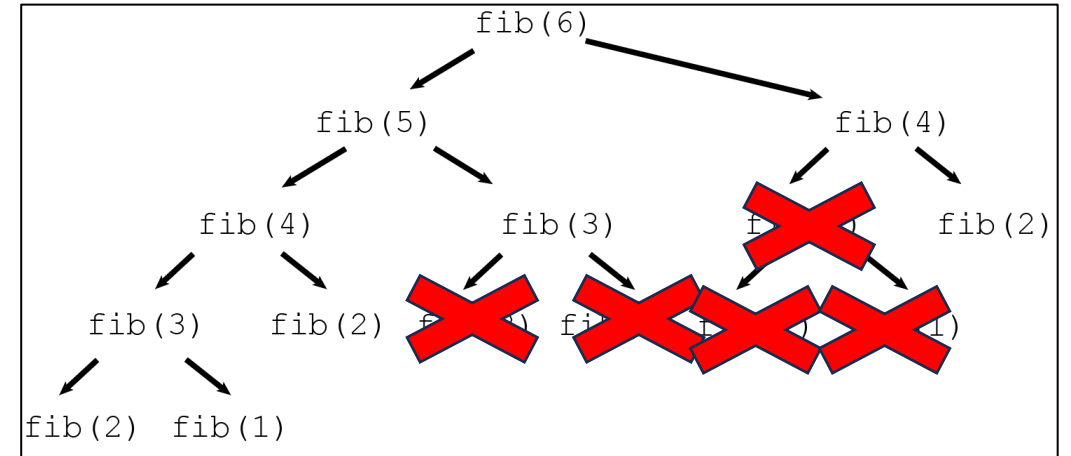
<Dynamic Programming>

메모이제이션을 수행하면, 한 번 계산한 값은 다시 계산하지 않으니 dp 배열을 채우는 시간만 고려하면 전체 시간복잡도가 됩니다.

따라서 메모이제이션을 수행하면 Bottom-up 방식과 같은 $O(N)$ 시간복잡도를 가집니다.



메모이제이션을 적용하지 않았을 때, $O(2^N)$



메모이제이션을 적용했을 때, $O(N)$

#동적 계획법

<Dynamic Programming>

점화식이 조금 다르게 생긴 문제를 풀어봅시다.

3 1로 만들기 (BOJ #1463)

<문제 설명>

- 수를 3으로 나누거나, 2로 나누거나, 1 뺄 수 있음
- 정수 N ($1 \leq N \leq 10^6$) 을 1으로 만들기 위해 필요한 연산 횟수의 최솟값 구하기

#동적 계획법

<Dynamic Programming>

예제를 살펴봅시다. 10을 1로 만들기 위해 몇 번의 연산이 필요할까요?

얼핏 보면 매 순간마다 가능한 가장 작은 수로 만드는 전략이 성립할 것 같습니다.

하지만 10을 2로 나누면 $10 - 5 - 4 - 2 - 1$ 4번의 연산이 필요한 반면, $10 - 9 - 3 - 1$ 처럼 3번의 연산으로도 가능합니다.

따라서 이러한 전략은 성립하지 않습니다.

#동적 계획법

<Dynamic Programming>

X에서 연산을 한 번 수행하면 $X-1$, $X/2$, $X/3$ 중 하나가 됩니다.

따라서 $X-1$, $X/2$, $X/3$ 각각을 1로 만드는 최소 연산 횟수를 모두 안다면 X를 1로 만드는 횟수 또한 알 수 있습니다.

X = 10일 때 1로 만드는 연산 횟수를 구해 봅시다.

$X-1 = 9$, $X/2 = 5$ 입니다. 9는 2번, 5는 3번의 연산으로 1로 만들 수 있습니다.

따라서 10에서 -1 연산을 수행하면 $2 + 1 = 3$ 번, /2 연산을 수행하면 $3 + 1 = 4$ 번의 연산으로 1로 만들 수 있습니다.

이때, $X-1$, $X/2$, $X/3$ 각각을 1로 만드는 횟수 또한 같은 방식으로 구할 수 있습니다.

예를 들어, $X-1$ 을 1로 만드는 횟수는 $(X-1)-1$, $(X-1)/2$, $(X-1)/3$ 을 1로 만드는 횟수를 안다면 구할 수 있습니다.

이러한 재귀 관계를 점화식으로 표현해 봅시다.

X를 1로 만드는 최소 연산 횟수를 $F(X)$ 라고 정의하면, $F(X) = \min(F(X-1), F(X/2), F(X/3)) + 1$ 으로 나타낼 수 있습니다.

#동적 계획법

<Dynamic Programming>

Bottom-up 방식으로 문제를 푸는 코드입니다.

```
#include <iostream>
#include <algorithm>
using namespace std;

int dp[1000005], n;
int main(){
    cin >> n;
    dp[1] = 0;
    for (int i = 2; i <= n; i++){
        dp[i] = dp[i-1] + 1;
        if (i % 2 == 0)
            dp[i] = min(dp[i], dp[i/2]+1);
        if (i % 3 == 0)
            dp[i] = min(dp[i], dp[i/3]+1);
    }
    cout << dp[n];
}
```

#동적 계획법

<Dynamic Programming>

Top-down으로 문제를 푸는 코드입니다.

```
#include <iostream>
#include <algorithm>
using namespace std;

int dp[1'000'005], N;

int f(int x){
    if (x == 1) return 0;
    if (dp[x]) return dp[x];
    dp[x] = f(x-1) + 1;
    if (x % 2 == 0)
        dp[x] = min(dp[x], f(x/2) + 1);
    if (x % 3 == 0)
        dp[x] = min(dp[x], f(x/3) + 1);
    return dp[x];
}

int main(){
    cin >> N;
    cout << f(N);
}
```

#동적 계획법

<Dynamic Programming>

문제를 재귀적으로 나누는 다른 예시를 살펴봅시다.

3 2*N 타일링 (BOJ #11726)

<문제 설명>

- 2*N 크기의 직사각형을 1*2 직사각형으로 꽉 채우는 경우의 수 구하기
- 경우의 수를 10007로 나눈 나머지를 출력해야 함
- $N \leq 1000$

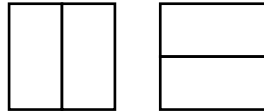
#동적 계획법

<Dynamic Programming>

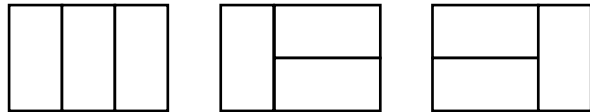
N = 1: 1가지



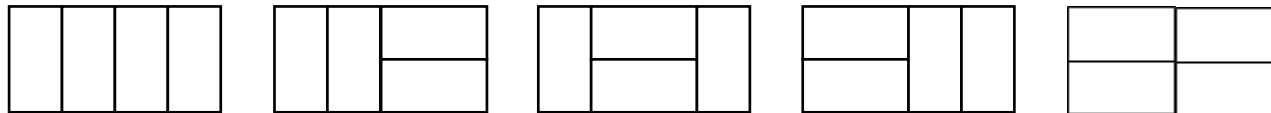
N = 2: 2가지



N = 3: 3가지



N = 4: 5가지



#동적 계획법

<Dynamic Programming>

1부터 4까지 답을 구해 보니 수상하게 **피보나치 수열**이 나옵니다..!

실제로 $2 \times N$ 직사각형을 채우는 경우의 수를 $f(N)$ 이라고 하면, 점화식은 $f(N) = f(N-1) + f(N-2)$ 입니다.

간단하게 설명하면, 가장 왼쪽 칸을 세로 블록으로 채우는 경우의 수가 $f(N-1)$, 가로 블록으로 채우는 경우의 수가 $f(N-2)$ 이기 때문입니다.

#동적 계획법

<Dynamic Programming>

Top-down으로 문제를 푸는 코드입니다.

문제에서 10007로 나눈 나머지를 출력하라는 이유는 답이 너무 커져 표현 가능한 범위를 넘기 때문입니다.

따라서 단순히 계산하고 출력할 때만 나머지를 취하면 이미 overflow가 난 상태이므로 틀리게 됩니다.

$(a + b) \% m = (a \% m + b \% m) \% m$ 이므로, 계산 중간중간마다 m 으로 나눈 나머지를 취해 주면 문제를 풀 수 있습니다.

```
#include <iostream>
using namespace std;

int dp[1005];

int f(int n){
    if (n == 1) return 1;
    if (n == 2) return 2;
    if (dp[n] != -1) return dp[n];
    dp[n] = (f(n-1) + f(n-2)) % 10007;
    return dp[n];
}

int main(){
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) dp[i] = -1;
    cout << f(n);
}
```

#필수 문제

3 1, 2, 3 더하기 (BOJ #9095)

PPT에서 해설한 문제입니다.

3 1로 만들기 (BOJ #1463)

PPT에서 해설한 문제입니다.

3 2^*N 타일링 (BOJ #11726)

PPT에서 해설한 문제입니다.

#연습 문제

3 파도반 수열 (BOJ #9461)

그림을 잘 보고 점화식을 세워 보세요.

2 이항 계수 2 (BOJ #11051)

조합 역시 점화식으로 나타낼 수 있습니다.

1 RGB거리 (BOJ #1149)

이항 계수 문제처럼 점화식에 두 개 이상의 변수가 들어갈 수도 있습니다.

3 돌 게임 3 (BOJ #9657)

턴제 게임은 DP로 나타낼 수 있습니다!
현재 상태에서 선공의 승리 여부를 DP로 나타내 보세요.

2 가장 긴 증가하는 부분 수열 (BOJ #11053)

$dp[i]$ = i 번째 수를 마지막 수로 가질 때 최대 길이
점화식을 어떻게 세워야 할까요?

1 포도주 시식 (BOJ #2156)

포도주를 지금까지 0잔 마신 경우, 1잔 마신 경우,
2잔 마신 경우를 분리해서 생각해 봅시다.



#연습 문제

5 동전 2 (BOJ #2294)

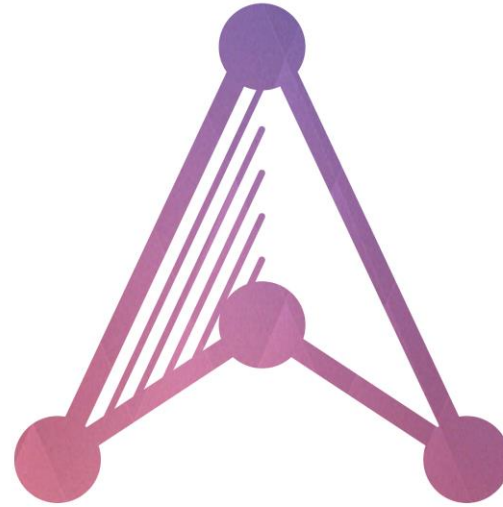
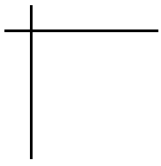
파이팅..!

4 동전 1 (BOJ #2293)

메모리 제한이 작아

2차원 배열을 사용하면 '메모리 초과'를 받습니다.

어떻게 해야 1차원 배열만으로 해결할 수 있을까요?



A L O H A
The algorithm club.

